

UNIT 8 · WEEKS 13–14

Networking: sockets

📅 Weeks 13–14 · dates to be confirmed </> Lab TD7 (solved + AI challenge) ~ 2 animations

LAB TD7 · SOLVED

Networking with sockets: from a minimal server to a web server with five models

A socket is used like a file (`read` , `write` , `close`), but it connects processes on different machines. This lab puts everything together: `fork` per client, one thread per connection and the producer–consumer of TD6 to serve many clients at once.

✓ **Assignment: lab report**

Hand in this lab with a short LaTeX report (just what is needed). What it must cover, the template, an example and the Prism guide are in the Assignment tab. Generative AI allowed, with a declaration.

EXERCISES

- | | |
|---|--|
| <p>1 The bare minimum of a server</p> <p>2 Echo server, iterative and with <code>fork</code></p> <p>3 Web server: iterative, one thread per connection and pool</p> | <p>4 Exploration: TCP states, the <code>listen</code> queue, the models under load and UDP</p> |
|---|--|

TD7 · 1

● Solved · tested

The bare minimum of a server

Statement. A program that waits for one connection (test it with `telnet` or `nc`) and ends: `socket` (IPv4, connection oriented) → `sockaddr_in` with `INADDR_ANY` and the port in network order (`htons`) → `bind` → `listen` → `accept` → `close` .

Solution



C tds/td7/sock.c

```

/*
 * TD7 - exercise 1: the server side of a TCP connection, step by step.
 * ./sock [port]      (default 7000)  then, from another terminal:
telnet localhost 7000
 * socket → bind → listen → accept → close.
 */
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <unistd.h>

#include "utils.h"

#define PORT 7000

int main(int argc, char *argv[])
{
    int port = argc > 1 ? atoi(argv[1]) : PORT;

    /* 1. An IPv4 (AF_INET), connected, reliable byte stream (SOCK_STREAM =
TCP). */
    int s = socket(AF_INET, SOCK_STREAM, 0);
    exit_if(s == -1, "socket");

    /* Allow restarting the server right away (otherwise bind fails for ~60
s: TIME_WAIT). */
    int yes = 1;
    exit_if(setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes) == -1,
"setsockopt");

    /* 2. Local address: any interface of this machine, the chosen port
(network byte order). */
    struct sockaddr_in addr = { 0 };
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons((uint16_t)port);

    /* 3. and 4. Bind the address, start listening (queue of 1 pending
connection). */
    exit_if(bind(s, (struct sockaddr *)&addr, sizeof addr) == -1, "bind");
    exit_if(listen(s, 1) == -1, "listen");
    printf("listening on port %d\n", port);

```

```
fflush(stdout);

/* 5. accept blocks until a client connects and returns a NEW descriptor
for it. */
int client = accept(s, NULL, NULL);
exit_if(client == -1, "accept");
printf("connection accepted, closing\n");

/* 6. Close the connection and the listening socket. */
exit_if(close(client) == -1 || close(s) == -1, "close");
return EXIT_SUCCESS;
}
```

Real output

```
$ ./sock 37634 & sleep 0.3; nc -z 127.0.0.1 37634 && echo 'client connected';
wait
listening on port 37634
connection accepted, closing
Connection to 127.0.0.1 37634 port [tcp/*] succeeded!
client connected
```

- `accept` returns a **new descriptor** for that client; the listening socket stays available to accept others.
- `htons` / `htonl`: the network uses *big-endian* order; x86 is *little-endian* (you saw it in TD1 with `od`).
- `SO_REUSEADDR` lets the server restart right away: without it, `bind` fails with `EADDRINUSE` while the previous connections are in `TIME_WAIT`.

[Animation A8.1 · Life cycle of a connection →](#)

TD7 · 2

● Solved · tested

Echo server, iterative and with `fork`

Statement. An echo server sends back to the client everything it receives, except `quit`, which closes the connection and waits for another one. Launch several clients at once: what happens? Change the server with `fork` to serve several of them.

Solution



C tds/td7/echo_server.c

```

/*
 * TD7 - exercise 2: echo server. Sends back every line it receives; "quit"
 closes the
 * connection and the server waits for the next client.
 * ./echo_server [port] [-f]      -f: one child process per client (fork)
 * Without -f the server is ITERATIVE: a second telnet waits until the first
 one leaves
 * (its connection stays in the listen queue). With -f each client gets its
 own process.
 */
#include <errno.h>
#include <netinet/in.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#include "utils.h"

#define PORT 7000
#define LINE 1024

/* Echo line by line until end of file or "quit". Returns when the client is
done. */
static void serve(int c)
{
    char line[LINE];
    size_t len = 0;

    for (;;) {
        ssize_t n = read(c, line + len, 1);          /* byte by byte: simple,
fine for a lab */
        if (n == -1 && errno == EINTR)
            continue;
        if (n <= 0)                                  /* 0: the client
closed; -1: error */
            return;
        if (line[len] == '\n' || len == LINE - 2) {

```

```

        len++;
        if (strncmp(line, "quit", 4) == 0)
            return;
        if (write_all(c, line, len) == -1)
            return;
        len = 0;
    } else {
        len++;
    }
}

}

int main(int argc, char *argv[])
{
    int port = PORT, use_fork = 0;
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "-f") == 0) use_fork = 1;
        else port = atoi(argv[i]);
    }

    int s = socket(AF_INET, SOCK_STREAM, 0);
    exit_if(s == -1, "socket");
    int yes = 1;
    exit_if(setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes) == -1,
"setsockopt");
    struct sockaddr_in addr = { .sin_family = AF_INET, .sin_port =
htonl((uint16_t)port),
                                .sin_addr.s_addr = htonl(INADDR_ANY) };
    exit_if(bind(s, (struct sockaddr *)&addr, sizeof addr) == -1, "bind");
    exit_if(listen(s, 8) == -1, "listen");
    printf("echo server on port %d (%s)\n", port, use_fork ? "one process per
client" : "iterative");
    fflush(stdout);

    /* Children that finish must not stay zombies: ignoring SIGCHLD makes the
kernel reap them. */
    if (use_fork)
        exit_if(signal(SIGCHLD, SIG_IGN) == SIG_ERR, "signal");

    for (;;) {
        int c = accept(s, NULL, NULL);
        if (c == -1 && errno == EINTR)
            continue;
        exit_if(c == -1, "accept");
    }
}

```

```

    if (!use_fork) {
        serve(c);
        exit_if(close(c) == -1, "close");
        continue;
    }
    pid_t pid = fork();
    exit_if(pid == -1, "fork");
    if (pid == 0) {
        exit_if(close(s) == -1, "close");           /* the child does not
accept */
        serve(c);
        _exit(EXIT_SUCCESS);
    }
    exit_if(close(c) == -1, "close");           /* the parent does not
talk to the client */
    }
}

```

Real output

```

$ python3 tcp_client.py echo 22751 'hola\nmundo\nquit'
sent: hola      received: hola
sent: mundo    received: mundo

```

Real output

```

$ python3 tcp_client.py blocked 22751      # iterative server
client B got nothing after 1 s: the server is busy with client A

```

Real output

```

$ python3 tcp_client.py blocked 34885      # server with fork
client B got 'hi\n' after 0.00 s

```

The iterative server serves one client at a time: while client A is connected, B waits in the `listen` queue and gets no answer (the test measures it: nothing in 1 s). With `-f`, every connection gets its own process and B gets its echo at once. The parent closes the client socket and the child the listening one; ignoring `SIGCHLD` avoids zombies. The inherited version detected `quit` with `strstr` on a buffer without terminator; it now reads line by line.

Web server: iterative, one thread per connection and a pool of 8 threads

Statement. The client (a browser) sends `GET file`; the server answers `HTTP/1.0 200 OK`, the MIME type and the content. Use `stat` to know whether it exists and its size, `mmap` to map it and `write` to send it. Then: (1) a new thread per connection; (2) 8 threads created at start that wait for connections — the server *produces* connections and the threads *consume* them.

Solution

C tds/td7/http_server.c

```
/* Handles one request on the connected socket c. Never exits the process. */
static void handle(int c)
{
    char req[REQUEST], path[1024], full[2048];
    ssize_t n = read(c, req, sizeof req - 1);
    if (n ≤ 0)
        return;
    req[n] = '\0';

    if (sscanf(req, "GET %1023s", path) ≠ 1) {
        reply_error(c, "400 Bad Request");
        return;
    }
    if (strstr(path, "..") ≠ NULL) { /* no escape from root */
        reply_error(c, "403 Forbidden");
        return;
    }
    if (strcmp(path, "/") = 0)
        strcpy(path, "/index.html");
    snprintf(full, sizeof full, "%s%s", root, path);

    struct stat st;
    if (stat(full, &st) = -1 || !S_ISREG(st.st_mode)) {
        reply_error(c, "404 Not Found"); /* PG204 answered
        "201" by mistake */
        return;
    }
    int fd = open(full, O_RDONLY);
    if (fd = -1) {
```

```

        reply_error(c, "403 Forbidden");
        return;
    }
    char head[256];
    int hn = snprintf(head, sizeof head,
                      "HTTP/1.0 200 OK\r\nContent-Type: %s\r\nContent-Length:
%lld\r\n\r\n",
                      mime_type(full), (long long)st.st_size);
    if (write_all(c, head, (size_t)hn) == -1) {
        close(fd);
        return;
    }
    if (st.st_size > 0) {
        /* mmap of 0 bytes is
an error */
        void *data = mmap(NULL, (size_t)st.st_size, PROT_READ, MAP_PRIVATE,
fd, 0);
        if (data != MAP_FAILED) {
            (void)write_all(c, data, (size_t)st.st_size); /* loops over
partial writes */
            munmap(data, (size_t)st.st_size);
        }
    }
    close(fd);
}

/* ----- thread per connection ----- */
static void *connection_thread(void *arg)
{
    int c = (int)(intptr_t)arg;
    handle(c);
    close(c);
    return NULL;
}

/* ----- pool: bounded queue of connected sockets ----- */
static int queue[QUEUE];
static int q_in = 0, q_out = 0;
static sem_t q_free, q_used;
static pthread_mutex_t q_lock = PTHREAD_MUTEX_INITIALIZER;

static void *worker(void *arg)
{
    (void)arg;
    for (;;) {
        sem_wait(&q_used);
        /* wait for a connection */

```

```
pthread_mutex_lock(&q_lock);
int c = queue[q_out];
q_out = (q_out + 1) % QUEUE;
pthread_mutex_unlock(&q_lock);
sem_post(&q_free);
handle(c);
close(c);
}
return NULL;
}
```

Real output

```
$ curl -s -i http://localhost:PORT/ | tr -d '\r' # mode pool
HTTP/1.0 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 20

<h1>FS0 · UTI</h1>
```

Real output

```
$ curl -s -o /dev/null -w '%{http_code}' ../missing.html ; curl --path-as-is
.../..etc/passwd
404
403
```

MODE	HOW IT SERVES	LIMIT
<code>-m iterative</code>	One client at a time	One slow client blocks everybody
<code>-m fork</code>	A child process per connection	Creating a process costs several times more than a thread (measured in $TD6 \cdot 5$)
<code>-m thread</code>	A new thread per connection (<code>pthread_detach</code>)	Thousands of connections = thousands of threads (memory, context switches)
<code>-m pool</code>	8 fixed threads + bounded queue (semaphores + mutex)	Bounded resources; 8 slow clients saturate it ($TD7 \cdot 4$)
<code>-m epoll</code>	One thread and an event loop: it only serves sockets that already have data	This simple version still writes blocking; nginx or Node.js also write without blocking

The tests send 40 concurrent requests (8 at a time) in every mode and check 40 answers 200 and a binary identical to the original. Minimal security: a path with `..` gets **403** (the client cannot leave the served folder) and what does not exist, **404**. The server ignores SIGPIPE so that it does not die if a client disconnects in the middle of the answer.

Fixed and completed

The inherited version answered `201 Not found` (the right code is 404), did not block `..`, sent no `Content-Length`, ignored partial writes, did not check the file open and lacked the thread and pool versions the statement asks for.

Animation A8.2 · Server models →

Exploration: what the kernel does with your sockets

Statement. (1) Read with `getsockopt(TCP_INFO)` the state of each socket at each step of a connection. (2) Find out what the `listen` backlog limits. (3) Measure the five modes of the web server with slow clients. (4) Compare TCP and UDP. (5) Check the `htons` bug of the original slides and resolve names with `getaddrinfo`. (6) Create a local socket.

1 · TCP states (`tcp_states.c`)

C tds/td7/tcp_states.c

```

    struct tcp_info ti;
    socklen_t len = sizeof ti;
    exit_if(getsockopt(s, IPPROTO_TCP, TCP_INFO, &ti, &len) == -1,
"getsockopt");
    return ti;
}

static const char *state(int s)
{
    if (s == -1)
        return "-";
    unsigned st = info(s).tcpi_state;
    return st < sizeof NAMES / sizeof NAMES[0] ? NAMES[st] : "?";
}

/* One line per step: the state of each socket. For a listening socket the
kernel reports the
 * accept queue in tcpi_unacked (connections waiting for accept) and
tcpi_sacked (backlog). */
static void show(const char *step, int l, int c, int a)
{
    struct tcp_info li = info(l);
    printf("%-26s | listener %s (queue %u/%u) | client %-11s | server %s\n",
        step, state(l), li.tcpi_unacked, li.tcpi_sacked, state(c),
state(a));
}

```

Real output

```
$ ./tcp_states PORT; ss -tn state time-wait '( dport = :PORT )'
socket+bind+listen(4)      | listener LISTEN (queue 0/4) | client -
| server -
connect (no accept yet)    | listener LISTEN (queue 1/4) | client ESTABLISHED
| server -
accept                     | listener LISTEN (queue 0/4) | client ESTABLISHED
| server ESTABLISHED
write/read                 | server read 6 bytes
client shutdown (FIN)     | listener LISTEN (queue 0/4) | client FIN_WAIT2
| server CLOSE_WAIT
server read                | server read returns 0: end of file
server shutdown (FIN)     | listener LISTEN (queue 0/4) | client CLOSED
| server CLOSED
                           | the client's end moved to a TIME_WAIT entry of
the kernel (60 s, see ss)
Recv-Q Send-Q Local Address:Port Peer Address:Port
0      0      127.0.0.1:42772    127.0.0.1:PORT
```

Three facts the output shows: the connection is **established before** **accept** (the kernel does the handshake; the queue goes to 1/4); whoever closes first goes through **FIN_WAIT2** while the other stays in **CLOSE_WAIT** until its **read** returns 0; and after the close the kernel keeps a **TIME_WAIT** entry (60 s) that **ss** shows even though the program has ended. That is why servers use **SO_REUSEADDR**.

2 · What the backlog limits (**backlog.c**)

Real output

```
$ ./backlog 4 8
listen(4), 8 clients connect
0.3 s, no accept yet:      accept queue 5 | clients ESTABLISHED 5,
SYN_SENT 3
accept 5 (total 5), wait:  accept queue 3 | clients ESTABLISHED 8,
SYN_SENT 0
accept 3 (total 8), wait: all 8 clients accepted
```

With **listen(fd, 4)** Linux completes **backlog + 1 = 5** handshakes and drops the SYN of the others, which stay in **SYN_SENT** and **resend** it (1 s, then 2 s, 4 s...). When the server accepts, the queue empties and the resends get in. The backlog does not limit

how many clients the server serves: it limits how many finished connections can wait for `accept`.

💡 For the curious: SYN cookies

Real output

```
$ nstat -n; ./backlog 2 8; nstat | grep -E
'Syncookies|ListenOverflows'
listen(2), 8 clients connect
0.3 s, no accept yet:      accept queue 3 | clients
ESTABLISHED 3, SYN_SENT 5
accept 3 (total 3), wait:  accept queue 3 | clients
ESTABLISHED 8, SYN_SENT 0
accept 3 (total 6), wait:  accept queue 0 | clients
ESTABLISHED 8, SYN_SENT 0
accept 0 (total 6), wait:  accept queue 0 | clients
ESTABLISHED 8, SYN_SENT 0
TcpExtSyncookiesSent 2
TcpExtListenOverflows 7
```

With `listen(fd, 2)` something subtler happens: when the queue of half-done handshakes also fills up, the kernel answers with **SYN cookies** (keeping no state, a defence against SYN floods). If the queue of accept is full when the final ACK arrives, that ACK is lost: the client believes it is ESTABLISHED, but the server does not know about it. The kernel counters (`nstat`) confirm it. That is why a real server's backlog is large (on this machine, `net.core.somaxconn = 4096`).

3 · The five models under load (`load.c`)



32 clients: first 1 (and then 8) that take 1 s to send their request, like a client on a bad network or a *slowloris* attack; 20 ms later, the fast ones. The latency of each group is measured, from `connect` to the whole reply.

Real output

```
$ ./http_server -m iterative & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
slow 1: median 1001 ms, max 1001 ms
fast 31: median 981 ms, max 982 ms
32 replies 200 OK of 32
slow 8: median 1001 ms, max 1001 ms
fast 24: median 982 ms, max 983 ms
```

Real output

```
$ ./http_server -m pool & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
slow 1: median 1001 ms, max 1001 ms
fast 31: median 1 ms, max 1 ms
32 replies 200 OK of 32
slow 8: median 1001 ms, max 1001 ms
fast 24: median 981 ms, max 981 ms
```

Real output

```
$ ./http_server -m epoll & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
slow 1: median 1001 ms, max 1001 ms
fast 31: median 1 ms, max 2 ms
32 replies 200 OK of 32
slow 8: median 1001 ms, max 1001 ms
fast 24: median 1 ms, max 2 ms
```

Iterative: a single slow client makes everybody wait ~1 s. Pool of 8: it copes with 1 slow client, but with 8 the eight threads are blocked in `read` and the same thing happens again. `fork`, thread per client and `epoll` answer the fast ones in a few milliseconds (the `fork` and `thread` outputs are in `tests/out/td7/`). The event loop does it with a **single thread**:

C tds/td7/http_server.c

```

/* ----- event loop: one thread watches every socket ----- */
static void event_loop(int s)
{
    int ep = epoll_create1(0);
    exit_if(ep == -1, "epoll_create1");
    struct epoll_event ev = { .events = EPOLLIN, .data.fd = s }, ready[64];
    exit_if(epoll_ctl(ep, EPOLL_CTL_ADD, s, &ev) == -1, "epoll_ctl");
    for (;;) {
        int n = epoll_wait(ep, ready, 64, -1);    /* sleeps until some socket
is readable */
        if (n == -1 && errno == EINTR)
            continue;
        exit_if(n == -1, "epoll_wait");
        for (int i = 0; i < n; i++) {
            int fd = ready[i].data.fd;
            if (fd == s) {                        /* a new connection: watch
it too */
                int c = accept(s, NULL, NULL);
                if (c == -1)
                    continue;
                ev.data.fd = c;
                exit_if(epoll_ctl(ep, EPOLL_CTL_ADD, c, &ev) == -1,
"epoll_ctl");
            } else {                            /* the request has arrived:
read won't wait */
                exit_if(epoll_ctl(ep, EPOLL_CTL_DEL, fd, NULL) == -1,
"epoll_ctl");
                handle(fd);
                close(fd);
            }
        }
    }
}

```

Animation A8.2 · Server models →

4 · TCP versus UDP (`udp.c`)



Real output

```
$ ./udp
TCP: 3 writes (one, two, three) → read #1 returns 11 bytes: "onetwothree"
UDP: 3 sendto (one, two, three) → recvfrom #1: 3 "one", recvfrom #2: 3
"two", recvfrom #3: 5 "three"
UDP: receive buffer 8192 bytes, 10000 sendto of 100 bytes succeeded, 8
received, 9992 lost
      kernel counter Udp RcvbufErrors grew by 9992
```

TCP is a **byte stream**: three `write`s arrive in a single `read` (as with a pipe), so the application protocol must mark where each message ends. UDP keeps each datagram's **boundaries**, but it does not wait for the receiver: with the buffer full, the kernel drops datagrams without telling anybody, even within the same machine, and only records it in `/proc/net/snmp`.

5 · Addresses and ports (`addr.c`) · 6 · Local socket (`local.c`)

Real output

```
$ ./addr 7000
port 7000 = 0x1b58; htons(7000) = 0x581b in memory order
sin_port = 7000:          the kernel bound port 22555
sin_port = htons(7000):   the kernel bound port 7000
getaddrinfo("localhost", "http"): IPv4 127.0.0.1 port 80
getaddrinfo("::1", "https"): IPv6 ::1 port 443
ip_unprivileged_port_start = 1024
sin_port = htons(80):     bind: Permission denied
sizeof: sockaddr 16, sockaddr_in 16, sockaddr_in6 28, sockaddr_un 110,
sockaddr_storage 128
```

Real output

```
$ ./local fso.sock
bind created fso.sock: a file of type socket (the 's' of ls -l), sun_path
holds 108 bytes
the server asks the kernel who connected: pid = mine, uid 1000 (my user)
read 5 bytes; after close the file stays until unlink
still there after close: yes
```

The example on slide 187 set `sin_port = 7000` without `htons`: on a *little-endian* processor the two bytes are read the other way round and the kernel opens port **22555**. `gethostbyname` (also on that slide) only knows IPv4, is not thread-safe and left POSIX in 2008: `getaddrinfo` resolves IPv4 and IPv6 with the same call. An `AF_UNIX` socket does not use 127.0.0.1 (that is `AF_INET` over the loopback interface): its address is a **file of type socket**, and the kernel tells the server who connected (`SO_PEERCRED`), something systemd, D-Bus or Docker use to decide permissions.

Animation A8.1 · Life cycle of a connection →

Checklist

Generative AI challenge · An `epoll` server that never blocks



Goal. Turn the `-m epoll` mode of `http_server.c` (which still reads and writes blocking) into a complete event loop, with non-blocking sockets and partial writes, and measure it against the thread-per-client mode with thousands of connections.

Starting prompt. Use it in the tool you prefer (Claude, ChatGPT, Gemini, Copilot, Codex...) with the *solved* version of the lab as context. Ask for a plan first and review every step before accepting it.

Prompt

I have an HTTP server in C with five modes; the `epoll` mode uses a single thread, but `handle()` does a read and a write that may block [paste `http_server.c` and `load.c`]. Help me: (1) use `accept4(..., SOCK_NONBLOCK)` and keep a state per connection (reading the request up to `\r\n\r\n`, writing the reply with an offset), registering `EPOLLOUT` only when `write` returns `EAGAIN`; (2) extend `load.c` to open 2000 connections that send very slowly; (3) compare the `epoll` mode with `-m thread` by measuring the latency of the fast clients, the number of threads (`ps -o nlwp`) and memory (`ps -o rss`). Explain the C10K problem to me and which OS resource limits each mode.

How to check the result

- A client that reads a big file very slowly (`curl --limit-rate 10k`) does not delay the others: `./load` shows it.
- `tests/run.sh td7` still passes and the binary file arrives identical (`cmp`).
- The table comes from your measurements and says on which machine they were made.
- `ulimit -n` explains at least one failure you observe when raising the concurrency.

Declare your use of AI: tool, relevant prompts, what you checked yourself and what you fixed. What counts is that you can explain and verify every decision.